

For task 3

Could you explain and elaborate more about the code how you realise the Candlestick and box plot? To be honest, you did not provide the core code and elaborate the code in detail
How your function works.

To create the static candlestick chart with mplfinance, I use the following code:

```
import talib

print("working")
from get_data import get_data
import mplfinance as fplt
import pandas as pd
import os
```

- Import talib for financial indicators calculation.
- Import get_data function that was developed in task 2, import mplfinance, pandas and os

```
ticker = 'CBA.AX'
feature_columns = ['Open', 'Close', 'High', 'Low', 'Volume']
start_date = '2021-01-01'
end_date = '2021-03-30'

# get data from get_data function
data_df = get_data(ticker, feature_columns, start_date, end_date,
save_data=True, split_by_date=True)
```

- Defined ticker, feature_columns, start date and end date and then call the get data function to get the stock market data to be plotted for a given ticker.

```
trade_days_month = 20
...
data_df["SMA"] = talib.SMA(*args: data_df.Close, timeperiod=trade_days_month)
data_df["RSI"] = talib.RSI(*args: data_df.Close, timeperiod=trade_days_month)
data_df["EMA"] = talib.EMA(*args: data_df.Close, timeperiod=trade_days_month)

print("TA-Lib Version : {}".format(talib.__version__))
```

- Calculate technical indicators like Simple Moving Average, Relative Strength Index and Exponential Moving Average which will later be plotted on the candlestick chart.

```
ema_plt = fplt.make_addplot(data_df["EMA"], color='red', width=1.2)

sma_plt = fplt.make_addplot(data_df["SMA"], color='blue', width=1.7)

rsi_plt = fplt.make_addplot(data_df["RSI"], color="grey", width=1.5,
                             ylabel="RSI",
                             secondary_y=True, linestyle='dashdot')
volume = fplt.make_addplot(data_df["Volume"], color="purple",)
```

- Using make_addplot method, create plot objects for ema, sma and rsi and volume. These plot objects will be used to make charts on top of the candle stick chart.

```
if not os.path.exists('images'):
    os.makedirs('images')
```

- This line of code creates a folder named images if it doesn't exist. Plots can be saved to this folder.

```
mc = fplt.make_marketcolors(
    up='tab:blue', down='tab:red',
    edge='lime',
    wick={'up': 'blue', 'down': 'red'},
    volume='lawngreen',
)
```

- Create a custom colour scheme for different movements of the market to make the chart easily interpretable.

```
s = fplt.make_mpf_style(base_mpl_style="ggplot", marketcolors=mc)
```

- Set custom style for the candlestick plot.

```
savefig = dict(fname=f"images/{ticker}.png", dpi=1000, pad_inches=0.75)
```

- Define parameters to save the image.

```
fplt.plot(
    data_df,
    type='candle',
    # Add extra plots on top of main candle stick chart. These are technical
    indicators.
    addplot=[sma_plt, sma_plt_scatter, ema_plt, rsi_plt, volume],
    style=s,
    title=ticker,
    ylabel='Price($)',
    ylabel_lower='shares\nTraded',
    volume=True,
```

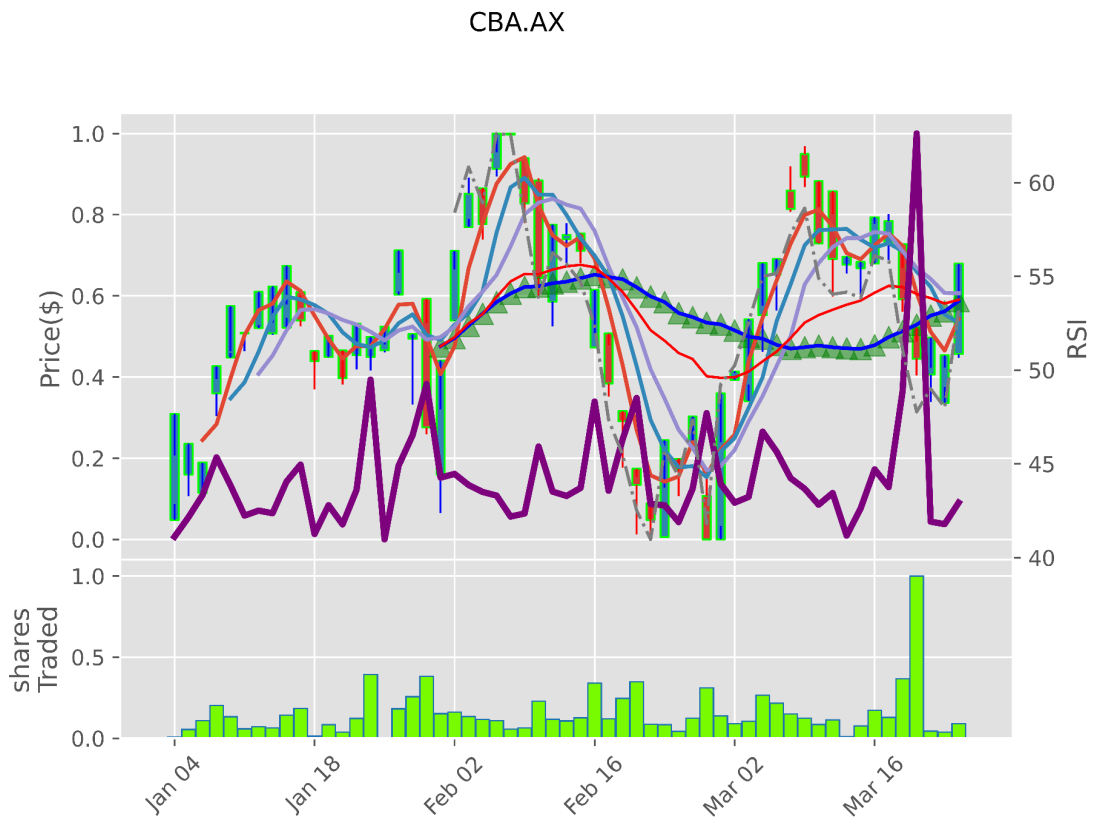
```

mav=(3, 5, 7),
# savefig='images/candlestick.png',
savefig=savefig,

show_nontrading=False
)

```

- Now plot the candle_stick chart



Final plot using mpl finance

Creating a interactable candlestick chart with plotly which can display data for $n \geq 1$ days;

```

print("working")
from get_data import get_data
# import mplfinance as fplt
import pandas as pd
import plotly.graph_objects as go

```

```
import plotly
import os
import plotly.io as pio
pio.renderers.default = "browser"
```

- Import plotly, pandas, get data, os and pio.
- Pio is used to set the default rendering platform as browser and not the compiler.

```
ticker = 'CBA.AX'
feature_columns = ['Open', 'Close', 'High', 'Low', 'Volume']
start_date = '2021-01-01'
end_date = '2021-03-30'
```

```
# get data from get_data function
data_df = get_data(ticker, feature_columns, start_date, end_date,
save_data=True, split_by_date=True)
```

- Agains define parameters and call the get_data function.

```
def plot_candlestick(data_df, num_days_aggregate=5):
```

- Notice the function takes a parameters called `num_days_aggregate` which allows a user to display candlestick for a specified number of trading days.

```
trade_days_month = 10

# resample the data to aggregate it for given number of days and choose the
first, max, min and last values for open, high, low and close respectively.
resampled_data = data_df.resample(f"{num_days_aggregate}D").agg({
    'Open': 'first',
    'High': 'max',
    'Low': 'min',
    'Close': 'last',
    'Volume': 'sum'
}).dropna()
```

- This code resamples the data for the specified number of days to be displayed as the candle stick and also drops missing values.. Note that user may choose to `num_days_aggregate = 1`.

```
resampled_data["SMA"] = talib.SMA(*args: resampled_data.Close, timeperiod=trade_days_month)
resampled_data["RSI"] = talib.RSI(*args: resampled_data.Close, timeperiod=trade_days_month)
resampled_data["EMA"] = talib.EMA(*args: resampled_data.Close, timeperiod=trade_days_month)
```

- Calculate technical indicators for the resampled data using talib.

```
candlestick = go.Candlestick(
    x=resampled_data.index,
    open=resampled_data['Open'],
    high=resampled_data['High'],
    low=resampled_data['Low'],
    close=resampled_data['Close']
)
```

- Create the candlestick chart

```
sma = go.Scatter(
    x=resampled_data.index,
```

```

        y=resampled_data["SMA"],
        # mode='lines',
        yaxis='y2',
        name='SMA'
    )
    ema = go.Scatter(
        x=resampled_data.index,
        y=resampled_data["EMA"],
        # mode='lines',
        yaxis='y1',
        name='EMA'
    )
    rsi = go.Scatter(
        x=resampled_data.index,
        y=resampled_data["RSI"],
        # mode='lines',
        yaxis='y1',
        name='RSI'
    )

```

- Create sma, ema and rsi plots to be overlaid on the main candlestick chart.

```

title = f"{ticker + start_date + end_date}"
print("Before go.Figure")

# create figure with candlestick, sma and ema
fig = go.Figure(data=[candlestick, sma, ema, rsi])

```

- Set title and define charts to be included in the final plot.

```

fig.update_layout(
    width=800,
    height=800,
    title=title,
    xaxis_title="Date",
    yaxis_title="Price",
    yaxis2=dict(
        title="Technical indicators",
        overlaying='y',
        side='right',
        position=0.15
    )
)

```

- Set the layout and additional y-axis axis for the technical indicators so that they don't skew the whole plot.

```
fig.show()
```

- Show the figure.

CBA.AX2021-01-012021-03-30



Final plot using plotly.

Box plot aggregating for n trading days.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from get_data import get_data

feature_cols = ['Open', 'Close', 'High', 'Low', 'Volume']

data = get_data('CBA.AX', feature_cols, '2021-01-01', '2021-02-06',
                save_data=True, split_by_date=True, )
import os
```

- Import seaborn, matplotlib.pyplot, os, pandas and define feature columns
- Get the data using the get_data function.

```
def calc_roll_stats(data_df, feature_columns, num_days_roll=5):
    # print("Calc roll stats")
    rolling_min =
data_df[feature_columns].rolling(window=num_days_roll).min().dropna()
    rolling_25 =
data_df[feature_columns].rolling(window=num_days_roll).quantile(0.25).dropna()
    rolling_median =
data_df[feature_columns].rolling(window=num_days_roll).median().dropna()
    rolling_75 =
data_df[feature_columns].rolling(window=num_days_roll).quantile(0.75).dropna()
    rolling_max =
data_df[feature_columns].rolling(window=num_days_roll).max().dropna()

    # aligned_index = rolling_min.index

    # print(f"rolling_min: {len(rolling_min)}")
    # print(f"rolling_25: {len(rolling_25)}")
    # print(f"rolling_median: {len(rolling_median)}")
    # print(f"rolling_75: {len(rolling_75)}")
    # print(f"rolling_max: {len(rolling_max)}")
    # print(f"Date: {len(data_df.index[num_days_roll - 1:])}")
    #
    # print(f"rolling_min after stacking: {len(rolling_min.stack())}")
    # print(f"rolling_25 after stacking: {len(rolling_25.stack())}")

    # concat the dataframes/series to create a single dataframe
    rolling_stats_df = pd.concat({
        "min": rolling_min,
        "25%": rolling_25,
        "median": rolling_median,
        "75%": rolling_75,
        "max": rolling_max
    })

    # print(f"rolling_data_df is: {rolling_stats_df} ")
    return rolling_stats_df
```

- Next calculate statistics of the given data for specified number of days.
- The statistics are: minimum, 25% quantile, median, 75% quantile and the maximum value.

```
rolling_data_reset = rolling_data_df.reset_index()
```

- Now reset the index for the data set that has statistics values. Basically we are converting the index(date) to a column, need to do it for use with seaborn plotting.

```
rolling_data_long = pd.melt(rolling_data_reset, id_vars=['level_0'],
                           value_vars=['Open', 'Close', 'High', 'Low',
'Volume'],
```

```
var_name='Stock Feature', value_name='Value')
```

- Pd.melt convert the dataframe into long format which is needed to plot with seaborn. Following example illustrates the melting process:

- Before melting the data looks like:

	Date	min_Open	min_Close	median_Open	median_Close
0	2021-01-01	100	105	105	110
1	2021-01-02	101	106	106	111
2	2021-01-03	102	107	107	112
3	2021-01-04	103	108	108	113
4	2021-01-05	104	109	109	114

- After melting the data looks like:

	Date	Stock Feature	Value
0	2021-01-01	min_Open	100
1	2021-01-02	min_Open	101
2	2021-01-03	min_Open	102
3	2021-01-04	min_Open	103
4	2021-01-05	min_Open	104

Now we move on top actual plot

```
plt.figure(figsize=(20, 16))
sns.boxplot(x='level_0', y='Value', hue='Stock Feature',
data=rolling_data_long)

# Enhancing the plot
plt.title(f'{num_days_roll} days rolling Boxplot', fontsize=16)
plt.xlabel('Statistic Type', fontsize=12)
plt.ylabel('Value', fontsize=12)
plt.xticks(rotation=45)
plt.legend(title='Stock Feature')
```

- Sets the size, sns.boxplot plots the actual chart for data = rolling_data_long then we set the title, x, y labels, ticks and the legend
- hue='Stock Feature' tells Seaborn to colour the boxes differently based on the stock feature (e.g., Open, Close, High, Low, Volume). It helps in distinguishing which stock feature each box belongs to.

```
if not os.path.exists('images'):
    os.makedirs('images')
plt.savefig('images/boxplot.pdf', dpi=1200)
plt.show()
```

- Create images if it does not exist, and then save the box plot in images folder
- Show the box plot.

Final box plot for 5 days window. The window size can be changed by the user by specifying the values for num_days_roll

